BOOMI REFERENCE DESIGN · HIGH-VOLUME ORDER INGEST

Parallel Order Pipeline

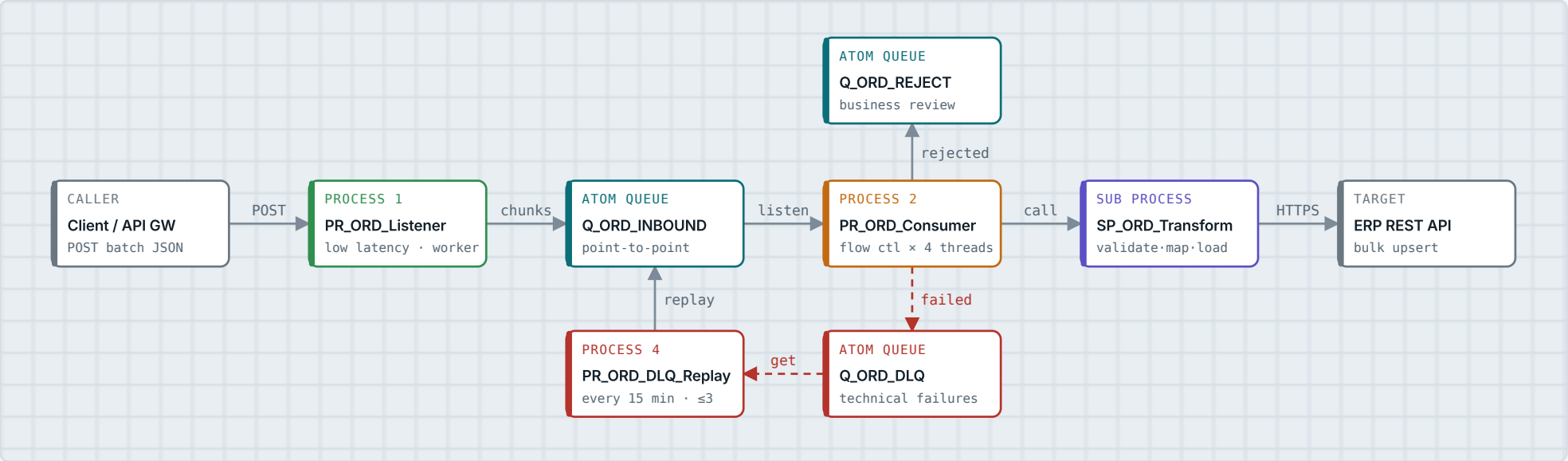
A four-process pattern for large payloads: a low-latency listener on Atom Workers accepts the batch and splits it onto an Atom Queue; a consumer fans the chunks out with Flow Control into a transform-and-load sub process; failures land in a dead-letter queue that a replay process drains with a retry cap.

Start · Logic / control · Data shaping · Connector · Process call · Error handling · Stop / return

---

00 · ARCHITECTURE

How the four processes connect



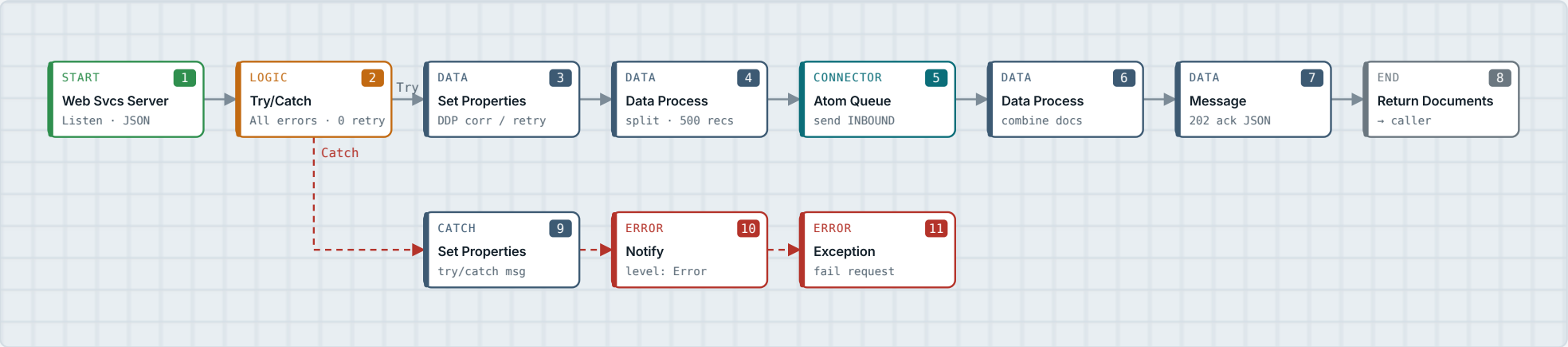
The queue between the listener and the consumer is what makes this hold up under load: the caller gets a fast 202, the heavy work runs on Molecule nodes at its own pace, and a crash mid-batch loses nothing because unacknowledged messages stay on the queue.

---

01 · PROCESS

PR_ORD_Listener

`Process Mode: Low Latency` `Runs on: Atom Worker (Runtime Cloud)` `Purge Data Immediately: On`



1 Start — Web Services Server

START

PROPERTY	VALUE	WHY
Type	`Connector`	Listener start
Connector	`Web Services Server`	REST/JSON endpoint on the cloud
Action	`Listen`	
Operation Type	`Execute · Object: orders`	Exposes /ws/rest/.../orders
Expected Input Type	`Single Data`	Whole batch arrives as one document
Response Output Type	`Single Data`	One ack JSON back
Request / Response profile	`JSON: ORD_Batch_Request / ORD_Ack`	Contract shown in API docs

2 Try/Catch

CONTROL

PROPERTY	VALUE	WHY
Failure Trigger	`All Errors`	Catch process + document errors
Retry Count	`0`	Listener must fail fast; caller retries

3 Set Properties — tag the batch

DATA

PROPERTY	VALUE	WHY
DDP_CORRELATION_ID	`Execution Property › Execution Id`	Trace across all 4 processes
DDP_SOURCE	`Static: "WEB_API"`	Origin for reporting
DDP_RETRY_COUNT	`Static: 0`	Read by DLQ replay
DDP_RECEIVED_AT	`Current Date (yyyy-MM-dd'T'HH:mm:ss)`	Age of failed items

> Use dynamic *document* properties here, not process properties. DDPs travel with each message through the Atom Queue; DPPs stay behind in this execution.

4 Data Process — split

DATA

PROPERTY	VALUE	WHY
Processing Step	`Split Documents`	
Split Type	`Split by Profile`	Structure-aware split
Profile Type / Profile	`JSON / ORD_Batch_Request`	
Link Element	`Root/Object/orders/Array/Element`	Repeating order node
Batch Count	`500`	Chunk size = unit of parallel work

5 Atom Queue — send

CONNECTOR

PROPERTY	VALUE	WHY
Action	`Send`	
Queue Name	`Q_ORD_INBOUND`	
Queue Type	`Point-to-Point`	Each chunk consumed exactly once
Connection	`None needed`	Uses the runtime's shared queue server

6 Data Process — combine

DATA

PROPERTY	VALUE	WHY
Processing Step	`Combine Documents`	Collapse N chunks back to 1
Profile	`JSON / ORD_Batch_Request`	

> Without this, the Message shape runs once per chunk and the Web Services Server returns multiple responses.

7 Message — ack body

DATA

PROPERTY	VALUE	WHY
Message	`{"status":"ACCEPTED","correlationId":"{1}"}`	Wrap literal braces in single quotes per Message shape rules
Variable {1}	`DDP_CORRELATION_ID`	Caller can query status

8 Return Documents

END

PROPERTY	VALUE	WHY
Label	(blank)	Top-level listener: sends response to caller

9 Set Properties — capture error

CATCH

PROPERTY	VALUE	WHY
DPP_ERR_MSG	Meta Information › Base › Try/Catch Message	Reason the batch failed

10 Notify

ERROR

PROPERTY	VALUE	WHY
Notification Level	Error	Shows in Process Reporting + alerts
Title	ORD listener rejected batch	
Message	Corr {1}: {2}	{1}=DDP_CORRELATION_ID, {2}=DPP_ERR_MSG

11 Exception

ERROR

PROPERTY	VALUE	WHY
Stop a single document	Off	Fail the whole request
Message	Ingest failed: {1}	Caller gets an HTTP error, not a false 202

01A · RUNTIME

Atom Worker and runtime settings

Atom Workers exist only on a Runtime Cloud (Boomi-hosted or private). A process set to Low Latency is executed by a pre-warmed worker JVM instead of being forked per call, which cuts startup cost to milliseconds.

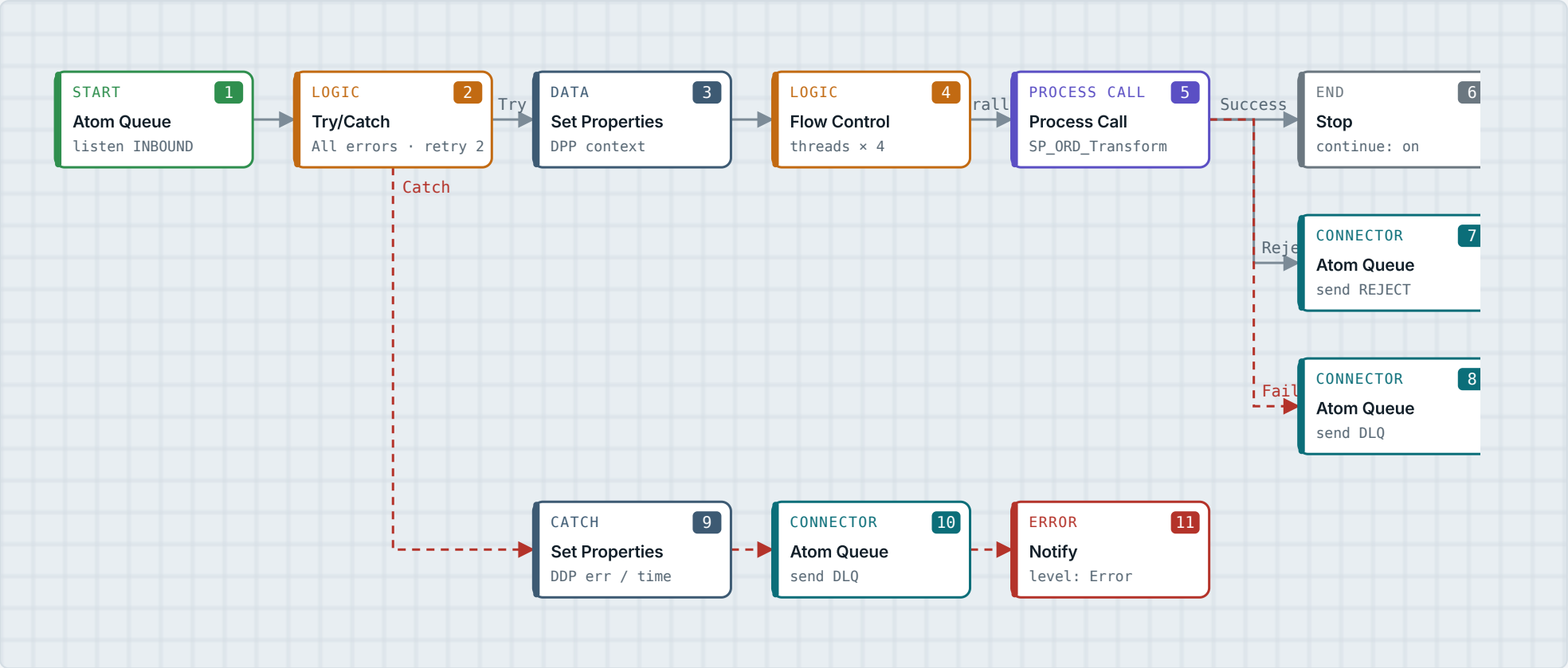
WHERE	SETTING	RECOMMENDATION
Build › Process options	`Process Mode = Low Latency`	Routes the listener to Atom Workers; no document tracking or Process Reporting docs in this mode
Runtime Cloud › Attachment Quotas	`Max Atom Workers`	Start at 2–4 per cloud node; raise when worker queue time grows
Runtime Cloud › Attachment Quotas	`Atom Worker max execution time`	Keep low (e.g. 30–60 s); the listener only splits and enqueues
Runtime Cloud › Attachment Quotas	`Worker heap / memory`	Size for the largest inbound batch × ~3 (raw + split + combined)
Runtime › Shared Web Server	`Authentication / API Management`	Basic or client certificate; put an API Gateway in front for throttling
Runtime › Atom Queue settings	`Shared queue server enabled, message store on persistent disk`	Queues survive node restarts

> Label names in the Runtime Cloud quota screen vary a little between platform releases; match on meaning if yours differ.

02 · PROCESS

PR_ORD_Consumer

Process Mode: General | Runs on: Molecule / cloud nodes | Allow Simultaneous Executions: On | Purge Data Immediately: On



1 Start — Atom Queue listen

START

PROPERTY	VALUE	WHY
Connector / Action	`Atom Queue / Listen`	Event-driven, no polling
Queue Name / Type	`Q_ORD_INBOUND / Point-to-Point`	
Batch size	`4–8 messages`	Feeds Flow Control with enough units to parallelise
Max concurrent executions	`2 per node`	Total parallelism = nodes × executions × threads

2 Try/Catch

CONTROL

PROPERTY	VALUE	WHY
Failure Trigger	`All Errors`	Includes thread / sub process crashes
Retry Count	`2`	Absorbs transient faults before DLQ

3 Set Properties — context

DATA

PROPERTY	VALUE	WHY
DPP_CORRELATION_ID	`Document Property › DDP_CORRELATION_ID`	Use in logs inside this execution
DPP_CONSUMER_EXEC	`Execution Property › Execution Id`	Join listener and consumer runs

4 Flow Control — parallel fan-out

CONTROL

PROPERTY	VALUE	WHY
Parallel Processing Type	`Threads`	Same JVM, low overhead; use Processes only for memory-heavy maps on a Molecule
Number of Units	`4`	≈ vCPUs per node; more threads just contend
Run Each Document Individually	`Off`	Each 500-row chunk already is the unit
Batch Count	`1`	One chunk per thread at a time

> Everything downstream of this shape runs once per unit. Process properties set inside one thread are not visible to the others, so pass per-document state as DDPs.

5 Process Call — sub process

EXEC

PROPERTY	VALUE	WHY
Process	`SP_ORD_Transform_Load`	Reusable, unit-testable
Wait for process to complete	`On`	Needed to get return paths
Abort if process fails	`Off`	Failure comes back on the Failed path instead of killing the parent
Return paths	`Success · Rejected · Failed`	Auto-created from the sub process's Return Documents labels

6 Stop — success

END

PROPERTY	VALUE	WHY
Continue processing other paths	`On`	Other branches still run

7 Atom Queue — reject queue

CONNECTOR

PROPERTY	VALUE	WHY
Action / Queue	`Send / Q_ORD_REJECT`	Bad data: never auto-retried, goes to business owners

8 Atom Queue — dead letter

CONNECTOR

PROPERTY	VALUE	WHY
Action / Queue	`Send / Q_ORD_DLQ`	Technical failure: eligible for replay

9 Set Properties — capture error

CATCH

PROPERTY	VALUE	WHY
DDP_ERR_MSG	`Meta Information › Base › Try/Catch Message`	Travels with the message to the DLQ
DDP_FAILED_AT	`Current Date`	
DDP_FAILED_STEP	`Static: "CONSUMER"`	Where in the pipeline it broke

10 Atom Queue — dead letter

CONNECTOR

PROPERTY	VALUE	WHY
Action / Queue	`Send / Q_ORD_DLQ`	Original chunk preserved for replay

11 Notify

ERROR

PROPERTY	VALUE	WHY
Notification Level	`Error`	Drives email / alert subscription
Message	`Corr {1} → DLQ: {2}`	

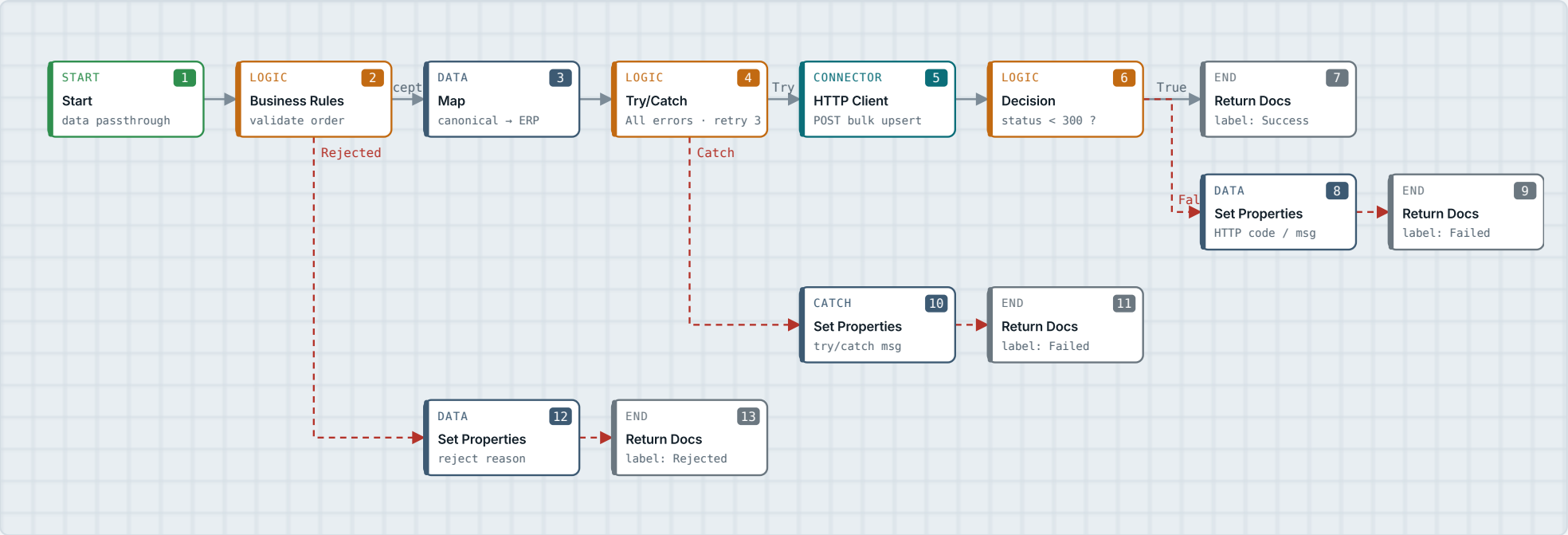
> End with Stop, not Exception: the chunk is safely parked in the DLQ, so the listener should acknowledge it rather than redeliver it forever.

---

03 · SUB PROCESS

SP_ORD_Transform_Load

Start: Data Passthrough | Mode: inherited from caller | 3 return labels



1 Start — Data Passthrough

START

PROPERTY	VALUE	WHY
Type	`Data Passthrough`	Receives the caller's documents and DDPs

2 Business Rules — validate

CONTROL

PROPERTY	VALUE	WHY
Profile	`JSON / ORD_Batch_Request`	
Rule 1	`orderId is not empty`	
Rule 2	`quantity > 0`	
Rule 3	`currency in (USD, EUR, GBP, INR)`	
Error message	`Per rule, e.g. "Missing orderId"`	Readable reason on the Rejected path

3 Map — canonical → ERP

DATA

PROPERTY	VALUE	WHY
Source / Destination	`ORD_Batch_Request / ERP_SalesOrder (JSON)`	
Lookups	`Cross Reference Table or Document Cache`	Never a DB call per row inside a parallel map

4 Try/Catch — target call

CONTROL

PROPERTY	VALUE	WHY
Failure Trigger	`All Errors`	Timeouts, connection refused
Retry Count	`3`	Boomi retries with increasing delay

5 HTTP Client — ERP upsert

CONNECTOR

PROPERTY	VALUE	WHY
Action	`Send · POST /salesorders/bulk`	One call per chunk
Content type	`application/json`	
Return Application Error Responses	`On`	4xx/5xx come back as documents for the Decision
Connection timeout / Read timeout	`10 000 / 120 000 ms`	Bounded, so a stuck thread can't hold the pool

6 Decision — status OK?

CONTROL

PROPERTY	VALUE	WHY
First value	`Meta Information › Base › Application Status Code`	
Comparison	`Less Than`	
Second value	`Static: 300`	True = 2xx

7 Return Documents — Success

END

PROPERTY	VALUE	WHY
Label	`Success`	Becomes a return path on the Process Call

8 Set Properties — HTTP failure

DATA

PROPERTY	VALUE	WHY
DDP_ERR_MSG	`Meta Information › Base › Application Status Message`	
DDP_HTTP_CODE	`Meta Information › Base › Application Status Code`	Replay logic can skip 4xx

9 Return Documents — Failed

END

PROPERTY	VALUE	WHY
Label	`Failed`	Shapes 9 and 11 share this label, so they merge into one return path

10 Set Properties — catch

CATCH

PROPERTY	VALUE	WHY
DDP_ERR_MSG	`Meta Information › Base › Try/Catch Message`	After 3 retries exhausted
DDP_FAILED_STEP	`Static: "ERP_CALL"`	

11 Return Documents — Failed

END

PROPERTY	VALUE	WHY
Label	`Failed`	Same label as shape 9

12 Set Properties — reject reason

DATA

PROPERTY	VALUE	WHY
DDP_REJECT_REASON	Business Rules result message (Rejected path)	Tells the data owner what to fix

13 Return Documents — Rejected

END

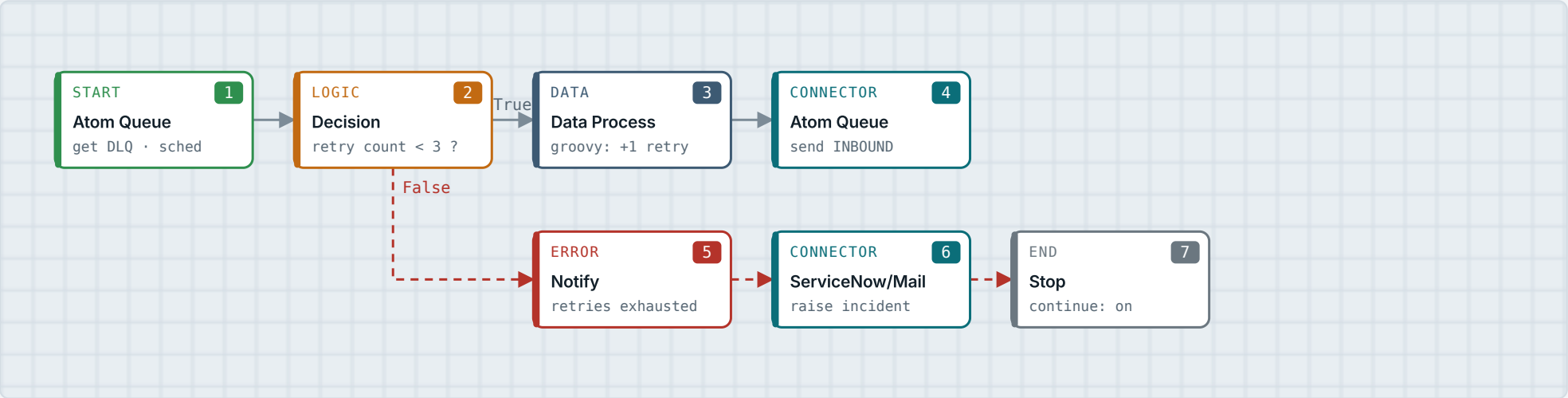
PROPERTY	VALUE	WHY
Label	Rejected	

---

04 · PROCESS

PR_ORD_DLQ_Replay

Schedule: every 15 min | Allow Simultaneous Executions: Off | Max 3 replays



1 Start — Atom Queue get

START

PROPERTY	VALUE	WHY
Connector / Action	`Atom Queue / Get`	Pull on schedule, gives a natural back-off
Queue	`Q_ORD_DLQ`	
Maximum documents	`200`	Avoid flooding the consumer on recovery

2 Decision — retry budget left?

CONTROL

PROPERTY	VALUE	WHY
First value	`Document Property › DDP_RETRY_COUNT`	
Comparison	`Less Than`	
Second value	`Static: 3`	Prevents poison-message loops

3 Data Process — increment retry

DATA

PROPERTY	VALUE	WHY
Processing Step	`Custom Scripting · Groovy 2.4`	Set Properties can't do arithmetic

```
import java.util.Properties
String KEY = "document.dynamic.userdefined.DDP_RETRY_COUNT"
for (int i = 0; i < dataContext.getDataCount(); i++) {
  InputStream is = dataContext.getStream(i)
  Properties props = dataContext.getProperties(i)
  int n = (props.getProperty(KEY) ?: "0") as int
  props.setProperty(KEY, String.valueOf(n + 1))
  dataContext.storeStream(is, props)
}
```

4 Atom Queue — replay

CONNECTOR

PROPERTY	VALUE	WHY
Action / Queue	`Send / Q_ORD_INBOUND`	Re-enters the normal parallel path

5 Notify — exhausted

ERROR

PROPERTY	VALUE	WHY
Notification Level	`Error`	
Message	`Corr {1} failed 3×: {2}`	{2} = DDP_ERR_MSG

6 Ticket / Mail connector

CONNECTOR

PROPERTY	VALUE	WHY
Connector	ServiceNow (Create Incident) or Mail (Send)	Human follow-up with payload attached

7 Stop

END

PROPERTY	VALUE	WHY
Continue processing other paths	On	

---

05 · EXCEPTION HANDLING

Five layers, each with one job

LAYER	MECHANISM	WHAT IT CATCHES → OUTCOME
1 · Data	Business Rules	Bad records → Rejected path → Q_ORD_REJECT. Never retried, because retrying bad data never fixes it.
2 · Call	Try/Catch (retry 3) around HTTP Client	Timeouts and blips → automatic retry, then Failed return path
3 · Response	Decision on Application Status Code	4xx/5xx the target returned → Failed with HTTP code kept as a DDP
4 · Process	Try/Catch (retry 2) around the Flow Control fan-out	Anything unexpected in a thread → Q_ORD_DLQ + Notify, and the queue message is acknowledged
5 · Recovery	Scheduled DLQ replay with a retry counter	Up to 3 replays, then an incident ticket. No silent loss, no infinite loop.

- Every message carries `DDP_CORRELATION_ID` from the listener onward, so one ID finds the request, its chunks, and any DLQ entry.
- The listener uses an Exception shape (caller must know it failed); the consumer uses Stop after the DLQ send (the chunk is safe, so redelivery would only duplicate it).
- Make the ERP call idempotent on `orderId`. Retries and replays then can't create duplicate orders.
- Subscribe to Error-level Notify events in Platform alerts, and add an Atom Queue depth alert on Q_ORD_DLQ.

---

06 · TUNING

Sizing the parallelism

- Effective concurrency = Molecule nodes × listener concurrent executions × Flow Control units. With 3 nodes × 2 × 4 you get 24 chunks in flight, about 12 000 orders.
- Check the ERP's rate limit first and size down to it. Parallelism above what the target accepts only turns into 429s and retries.
- Chunk size trades memory against call count. 200–1 000 records per chunk suits most JSON APIs; drop it for wide records.
- Turn on Purge Data Immediately for the consumer and listener. Disk I/O on document storage is often the real bottleneck at volume.
- Choose Flow Control *Processes* instead of Threads only when a single chunk's map is large enough to pressure one JVM's heap, and only on a Molecule or Runtime Cloud.